Jeudi 19/11, après-midi — Trier sans modifier le tableau reçu : toSorted

Kevin Bardoux

jeudi 19 novembre 2026

Table des matières

Objectifs 2
Après-midi — Trier sans modifier 2
Trier avec toSorted (20 min) 2
Trier, en image : avant / après 3
Trier, l'exemple minimal 3
Trier — attention et vérification 4
Pas à pas ensemble — Le palmarès des musées (40 min) 4
Variante à faire seuls 7
Enregistrer et envoyer (2 min) 7
Exercices — Trier sans modifier 7
Palier 1 — Trier des prix (25 min) 7
Lire une documentation en anglais — *Array.prototype.sort()* (15 min) 8
Palier 2 — Le palmarès (45 min) 8
Palier 3 — Trier des noms accentués (35 min) 9
Enregistrer et envoyer (2 min) 10
À retenir 10
Si le temps le permet 10
Débrief 11
Sources 11

Note

Compétences visées : C4 · communiquer en anglais · résolution de problème **Livrable** : le pas à pas `pas-a-pas-tri` envoyé sur GitHub, `node --test t1-trier-prix.test.js` au vert, et vos quatre réponses de lecture en anglais.

! Le minimum du jour

- Le pas à pas « Le palmarès des musées », fait avec le groupe.
- Le palier 1 : `node --test t1-trier-prix.test.js` au vert.
- La lecture en anglais : les quatre réponses écrites.

Les paliers 2 et 3, puis la rubrique « Si le temps le permet », sont la suite naturelle, pour ceux qui avancent.

Objectifs

- Trier un tableau de nombres avec `toSorted` et une fonction de comparaison.
- Trier un tableau d'objets selon une propriété.
- Expliquer pourquoi on ne modifie pas le tableau reçu.

Comment se passe l'après-midi. Une seule notion : trier sans modifier. Puis, à 14 h 15, un quart d'heure de lecture en anglais, sur la page de documentation de la méthode du jour.

Après-midi — Trier sans modifier

Trier avec toSorted (20 min)

- **Pourquoi** : un palmarès, une liste par prix, sans abîmer les données.
- **`toSorted`** : une copie triée ; le tableau reçu ne change pas.
- **`(a, b) => a - b`** : du plus petit au plus grand.

Pourquoi. Les musées du plus visité au moins visité, des produits du moins cher au plus cher : il faut trier. Mais le tableau reçu sert peut-être ailleurs (à afficher la liste dans son ordre d'origine, par

exemple) : le trier « sur place » le modifierait pour tout le reste du programme.

Définition. `toSorted` renvoie un **nouveau** tableau, trié ; le tableau d'origine n'est pas modifié (MDN Web Docs, s. d.-b). Sans argument, il convertit les éléments en textes et les compare caractère par caractère. Avec une fonction de comparaison `(a, b) => …`, il range `a` devant `b` quand la fonction renvoie un nombre négatif, derrière quand elle renvoie un nombre positif (MDN Web Docs, s. d.-a) : `(a, b) => a - b` range du plus petit au plus grand, `(a, b) => b - a` du plus grand au plus petit.

Trier, en image : avant / après

	`prices` avant	ce qui est renvoyé	`prices` après
`prices.toSorted()`	`[25, 8, 100, 10]`	`[10, 100, 25, 8]` (ordre des textes)	`[25, 8, 100, 10]`
`prices.toSorted((a, b) => a - b)`	`[25, 8, 100, 10]`	`[8, 10, 25, 100]`	`[25, 8, 100, 10]`
`prices.sort((a, b) => a - b)`	`[25, 8, 100, 10]`	`[8, 10, 25, 100]`	**`[8, 10, 25, 100]`** : modifié

Les deux premières lignes laissent `prices` intact ; la troisième, avec `sort`, le modifie. La première ligne montre l'autre piège : sans fonction de comparaison, `100` passe devant `25`, comme « 100 » devant « 25 » dans un dictionnaire.

Trier, l'exemple minimal

```
const prices = [25, 8, 100, 10];

const cheapFirst = prices.toSorted((a, b) => a - b);
// [ 8, 10, 25, 100 ]

const museums = [{ name: 'A', total: 5 }, { name: 'B',
total: 9 }];
const ranking = museums.toSorted((a, b) => b.total -
```

```
a.total);
// B d'abord : 9 visites


console.log(prices); // [ 25, 8, 100, 10 ] : inchangé
```

Trier — attention et vérification

> ⚠ Attention
>
> `sort` trie le tableau **lui-même** : après `prices.sort(…)`, `prices` est modifié pour tout le programme. Pour ne pas modifier le tableau reçu, utilisez `toSorted`. Et donnez toujours une fonction de comparaison pour des nombres : sans elle, `100` passe devant `25`.

- **Vérifiez** : que donne `[3, 20, 100].toSorted()` ?
- Et `[3, 20, 100].toSorted((a, b) => b - a)` ?

Pas à pas ensemble — Le palmarès des musées (40 min)

- Le formateur code, vous tapez en même temps, une étape à la fois.
- Code et commentaires : `//` au-dessus de chaque ligne.
- Dossier : `pas-a-pas-tri`, fichier `palmares.js`.
- Perdu ? Dossier `rattrapage/tri/`, une étape à la fois.

Palier 1 : guidé. Vérifiez d'abord votre version de Node : `node -v` doit afficher `v20` ou plus (`v24…` depuis la semaine 2). Ouvrez `pas-a-pas-tri/palmares.js` : les visites de six musées du Grand-Est en 2019 (Ministère de la Culture, s. d.). Dans le terminal : `cd pas-a-pas-tri`.

Avant de commencer. Lancez `node palmares.js`, depuis `pas-a-pas-tri`. **Vous devez voir** : les données de départ, sans erreur. Chaque étape ajoute ses lignes à la fin du fichier, et sa sortie à la fin de la console.

Étape 1 — Trier sans fonction de comparaison. À la fin du fichier, triez `visits` avec `toSorted()`, sans argument.

Avant d'exécuter : dans quel ordre les nombres vont-ils sortir ?

```
// toSorted sans argument : trie comme des textes
const asText = visits.toSorted();
// Surprise : 179932 arrive juste après 1207
console.log(asText);
```

Enregistrez, puis lancez `node palmares.js` dans le terminal, depuis `pas-a-pas-tri`.

Vous devez voir, à la fin de la sortie :

```
[ 1207, 179932, 29212, 3165, 3808, 51775 ]
```

Perdu ? Copiez `rattrapage/tri/etape-1/palmares.js` à la place de votre `palmares.js`, puis repartez avec le groupe.

Étape 2 — Comparer des nombres. Donnez à `toSorted` la fonction `(a, b) => a - b`.

Avant d'exécuter : `1207` ou `179932` en premier ?

```
// Comparaison : a - b range du plus petit au plus
grand
const ascending = visits.toSorted((a, b) => a - b);
// Affiche les visites dans l'ordre croissant
console.log(ascending);
```

Enregistrez, puis lancez `node palmares.js` dans le terminal, depuis `pas-a-pas-tri`.

Vous devez voir, à la fin de la sortie :

```
[ 1207, 3165, 3808, 29212, 51775, 179932 ]
```

Perdu ? Copiez `rattrapage/tri/etape-2/palmares.js` à la place de votre `palmares.js`, puis repartez avec le groupe.

Étape 3 — Le palmarès, et l'original. Triez du plus grand au plus petit, puis affichez `visits`.

Avant d'exécuter : `visits` est-il trié, maintenant ?

```
// b - a : du plus grand au plus petit
const descending = visits.toSorted((a, b) => b - a);
// Affiche le palmarès des visites
console.log(descending);
// toSorted n'a pas modifié le tableau reçu
console.log(visits);
```

Enregistrez, puis lancez `node palmares.js` dans le terminal, depuis `pas-a-pas-tri`.

Vous devez voir, à la fin de la sortie :

```
[ 179932, 51775, 29212, 3808, 3165, 1207 ]
[ 3165, 29212, 51775, 3808, 1207, 179932 ]
```

Perdu ? Copiez `rattrapage/tri/etape-3/palmares.js` à la place de votre `palmares.js`, puis repartez avec le groupe.

Étape 4 — Trier des objets. Créez `museums`, triez-les selon `total`, puis gardez leurs noms avec `map`.

Avant d'exécuter : quel musée va arriver en tête ?

```
// Tableau d'objets : trois musées
const museums = [
  // Chaque élément : un objet { name, total }
  { name: 'musée Arthur Rimbaud', total: 29212 },
  { name: "écomusée d'Alsace", total: 179932 },
  { name: "musée de l'Ardenne", total: 51775 },
];
// Trie les objets selon leur champ total
const ranking = museums.toSorted((a, b) => b.total -
a.total);
// map : les noms, du plus visité au moins visité
console.log(ranking.map((item) => item.name));
```

Enregistrez, puis lancez `node palmares.js` dans le terminal, depuis `pas-a-pas-tri`.

Vous devez voir, à la fin de la sortie :

```
[ "écomusée d'Alsace", "musée de l'Ardenne", 'musée Arthur Rimbaud' ]
```

Perdu ? Copiez `rattrapage/tri/etape-4/palmares.js` à la place de votre `palmares.js`, puis repartez avec le groupe.

Variante à faire seuls
Pour ceux qui ont validé l'étape avant les autres, dans le même fichier. Affichez les noms du moins visité au plus visité. **Réussi si** : la console affiche `[ 'musée Arthur Rimbaud', "musée de l'Ardenne", "écomusée d'Alsace" ]`.

Enregistrer et envoyer (2 min)
Depuis `~/dwwm/dwwm-tp/semaine-07` : `git status`, `git add pas-a-pas-tri`, `git commit -m "Ajoute le pas à pas tri"`, puis `git push`. **Réussi si** : `git status` affiche `rien à valider, la copie de travail est propre`.

Exercices — Trier sans modifier

Palier 1 — Trier des prix (25 min)

1. `t1-trier-prix.js` : du moins cher au plus cher, avec `toSorted`.
2. Le tableau reçu ne doit pas changer.
3. **Réussi si** : `node --test t1-trier-prix.test.js` au vert.

Palier 1 : guidé. Durée : 25 min.

1. Ouvrez `t1-trier-prix.js`. `sortPrices(prices)` renvoie un nouveau tableau, du prix le plus bas au plus haut.
2. Écrivez `return prices.toSorted((a, b) => a - b);`.
3. Lancez `node --test t1-trier-prix.test.js`. Le test « des nombres, pas des textes » échoue si vous oubliez la fonction de comparaison ; le test « le tableau reçu n'est pas modifié » échoue avec `sort`.
4. Un commit.

Réussi si : `node --test t1-trier-prix.test.js` affiche `ℹ fail 0`.

> **i** Aides
>
> - **Le cours** : Jeudi après-midi, « Trier avec toSorted » ; le pas à pas, étape 2.
> - **L'IA** : CS50.ai (palier IA 1) : demandez ce que renvoie `a - b` quand `a` est plus petit que `b`, pas le code.
> - **MDN** : Array.prototype.toSorted().

Lire une documentation en anglais — *Array.prototype.sort()* (15 min)

- Une page de MDN, en anglais, sans traduction.
- Quatre questions ; réponses en français ou en anglais.
- Sans IA : comme au questionnaire professionnel.

> **!** Sans IA
>
> Ni IA, ni traduction automatique : lisez la page en anglais, comme au questionnaire professionnel du titre. Un mot inconnu ? Devinez-le par le contexte, puis vérifiez après la lecture.

Ouvrez la page Array.prototype.sort() de MDN, en anglais (MDN Web Docs, s. d.-c). Lisez l'introduction, puis la partie *Parameters*, sous-partie `compareFn`. Répondez sur votre feuille, en une phrase chacune.

1. What happens to the elements when `compareFn` is omitted?
2. What does a negative return value of `compareFn` mean?
3. What does a return value of zero mean?
4. Which method does the page recommend to sort an array without changing the original array?

Réussi si : quatre réponses écrites, chacune avec un mot ou une expression de la page qui la justifie.

Palier 2 — Le palmarès (45 min)

1. `dc5-top.js` : les `n` musées les plus visités.
2. Trier des objets, puis garder les `n` premiers.
3. **Réussi si** : `node --test dc5-top.test.js` au vert.

Palier 2 : recommandé, semi-guidé. Durée : 45 min.

1. Ouvrez `dc5-top.js`. `topMuseums(museums, n)` renvoie les `n` musées les plus visités (champ `total`), du plus au moins visité, sans modifier le tableau reçu.
2. Triez les objets avec `toSorted` et une comparaison sur `total`, du plus grand au plus petit (le pas à pas, étape 4).
3. Étape documentation : dans MDN, page *Array*, trouvez la méthode qui renvoie un **nouveau** tableau avec une partie des éléments (du début jusqu'à une position), sans modifier l'original. Appliquez-la au tableau trié.
4. Lancez `node --test dc5-top.test.js`.

Réussi si : `node --test dc5-top.test.js` affiche `ℹ fail 0`. Le dernier test lit le vrai fichier, déjà nettoyé (`data/musees-grand-est-2019-propres.json`) : le premier du palmarès est l'écomusée d'Alsace.

> ℹ Aides
>
> - **Le cours** : Jeudi après-midi, « Trier avec toSorted » ; le pas à pas, étape 4.
> - **L'IA** : CS50.ai (palier IA 1) : demandez comment comparer deux objets selon une propriété, pas le code.
> - **MDN** : Array, liste des méthodes ; Array.prototype.toSorted().

Palier 3 — Trier des noms accentués (35 min)

- `t2-trier-noms.js` : l'ordre alphabétique français.
- « Échirolles » entre « Dinan » et « Zillisheim ».
- **Réussi si** : `node --test t2-trier-noms.defi.test.js` au vert.

Palier 3 : autonome. Durée : 35 min.

`sortNames(names)` renvoie un nouveau tableau de noms dans l'ordre alphabétique **français** : avec `toSorted()` seul, « Échirolles » arrive après « Zillisheim », et « Musée C » avant « musée A » : les majuscules passent devant. Trouvez dans MDN la méthode des chaînes de caractères qui compare deux textes

selon une langue, et servez-vous-en dans la fonction de comparaison.

Réussi si : `node --test t2-trier-noms.defi.test.js` affiche `ℹ fail 0`.

> ℹ Autonome
>
> Vous allez bloquer : c'est voulu, c'est le métier. Méthode : 1. lisez le message d'erreur en entier ; 2. cherchez-le dans la fiche « Lire un message d'erreur » ; 3. lisez la documentation ; 4. après 15 minutes bloqué sur le même point, demandez.

> ℹ Aides
>
> - **Le cours** : Jeudi après-midi, « Trier avec toSorted » ; la lecture en anglais, partie *compareFn*.
> - **L'IA** : CS50.ai (palier IA 1) : demandez pourquoi « É » est rangé après « Z », pas le code.
> - **MDN** : String, liste des méthodes.

Enregistrer et envoyer (2 min)
Même rituel : `git status`, `git add`, `git commit -m "…"`, `git push`, depuis `semaine-07`. C'est la fin de la demi-journée : le `git push` est obligatoire.

À retenir

> À retenir
>
> - **`toSorted`** : une copie triée ; `sort` modifie le tableau lui-même.
> - **Des nombres** : toujours une fonction de comparaison, `(a, b) => a - b`.
> - **Des objets** : comparer une propriété, `(a, b) => b.total - a.total`.

Si le temps le permet

- Les paliers 2 et 3 non terminés.
- Le parcours d'autonomie, à partir de la ligne 4.

Débrief

- Pourquoi `100` passe-t-il devant `25` sans fonction de comparaison ?
- Quelle différence entre `sort` et `toSorted` ?

Sources

MDN Web Docs. s. d.-a. « Array : méthode sort() - JavaScript ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/sort.

MDN Web Docs. s. d.-b. « Array : méthode toSorted() - JavaScript ». Consulté le 10 novembre 2026. https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array/toSorted.

MDN Web Docs. s. d.-c. « Array.prototype.sort() - JavaScript ». Consulté le 10 novembre 2026. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort.

Ministère de la Culture. s. d. « Fréquentation des Musées de France (data.gouv.fr) ». Consulté le 26 septembre 2026. https://www.data.gouv.fr/datasets/frequentation-des-musees-de-france-1.